

# Entrega Contínua de uma Plataforma de Pedidos em Microsserviços

Relatório Técnico — Parte Teórica | Projeto Pedidos Veloz | Cloud DevOps Engineer

## 1. Introdução e contexto do desafio

A Loja Veloz é uma empresa de varejo digital de médio porte que, após 12 meses de crescimento acelerado, passou a sofrer com indisponibilidades durante deploys, dificuldade de escalar em picos de acesso e baixa rastreabilidade de falhas entre serviços. O cenário técnico atual — cada time sobe partes do sistema "como dá", em máquinas diferentes — é sintoma direto da ausência de práticas de DevOps e de uma arquitetura preparada para escala. Este relatório apresenta a fundamentação teórica que embasa a proposta prática de modernização da plataforma "Pedidos Veloz", composta por API Gateway, Serviço de Pedidos, Serviço de Pagamentos, Serviço de Estoque e banco de dados PostgreSQL, em preparação para uma campanha promocional com pico de tráfego previsto.

## 2. Arquitetura de microsserviços e o papel do DevOps cloud-native

Arquitetura de microsserviços é um estilo arquitetural que decompõe uma aplicação em serviços pequenos, independentemente implantáveis, organizados em torno de capacidades de negócio (no caso da Pedidos Veloz: pedidos, pagamentos e estoque), que se comunicam por APIs bem definidas — em vez de um único monólito. Essa decomposição traz autonomia de deploy e de escala por serviço, mas desloca a complexidade do código para a operação: agora existem múltiplos processos, redes, falhas parciais e versões a coordenar. É exatamente aí que entra o DevOps: um conjunto de práticas culturais e técnicas — integração e entrega contínuas, automação de infraestrutura, observabilidade e cultura de responsabilidade compartilhada entre desenvolvimento e operação — que existe para tornar essa complexidade administrável. Em ambientes cloud-native, o DevOps se apoia em três pilares práticos: containers como unidade padronizada de empacotamento, orquestração para gerenciar o ciclo de vida desses containers em escala, e pipelines automatizados que levam código a produção com testes e validações antes de qualquer deploy.

Um guia amplamente adotado para construir aplicações adequadas a esse modelo é a metodologia **12-Factor App**, que orienta, entre outros pontos: configuração armazenada em variáveis de ambiente (não em código), logs tratados como fluxos de eventos (event streams) enviados a stdout, processos stateless e sem afinidade a disco local, e disposability — processos que sobem e descem rapidamente e de forma segura. Esses princípios foram aplicados diretamente na implementação da Pedidos Veloz (ver relatório técnico/prático) e são o que torna os serviços compatíveis com orquestração automática e escala horizontal.

## 3. Containerização: Docker e Kubernetes — diferenças e quando usar cada um

Docker resolve o problema de empacotamento: transforma uma aplicação e suas dependências em uma imagem imutável e portátil, que roda de forma idêntica no notebook do desenvolvedor e em produção. Docker Compose estende isso para múltiplos containers relacionados, permitindo declarar redes, volumes e variáveis de ambiente de todo um sistema multi-serviço e subi-lo com um único comando — adequado para desenvolvimento local e ambientes de baixa escala, mas sem mecanismos nativos de auto-cura, escalonamento automático ou distribuição entre múltiplas máquinas físicas.

Kubernetes resolve o problema seguinte: orquestrar esses containers em escala e em produção. Ele adiciona reconciliação contínua do estado desejado (se um Pod morre, outro sobe automaticamente), escalonamento horizontal (HPA), balanceamento de carga interno via Services, configuração desacoplada via ConfigMaps/Secrets, estratégias de deploy controladas (rolling update nativo) e políticas

de segurança e rede (Pod Security Admission, NetworkPolicies). A regra prática adotada neste projeto: Docker/Compose para o ciclo de desenvolvimento local — rápido, simples, sem exigir acesso a um cluster — e Kubernetes para produção, onde disponibilidade, escala sob demanda e auto-recuperação são requisitos de negócio (essenciais durante a campanha promocional da Loja Veloz).

## 4. Fundamentação teórica

### 4.1 Orquestração de containers

Um orquestrador como o Kubernetes gerencia o ciclo de vida completo dos containers: agendamento de Pods em nós disponíveis, health checks via probes de liveness/readiness (que determinam se um container deve ser reiniciado ou temporariamente removido do balanceamento), auto-scaling horizontal (HPA) e vertical (VPA), e primitivas de rede e armazenamento persistente. A unidade mínima de implantação é o Pod; Deployments gerenciam réplicas e estratégias de atualização; Services expõem um endpoint estável independente de quais Pods estão vivos no momento.

### 4.2 CI/CD em ambientes distribuídos

Integração Contínua (CI) automatiza a validação de cada mudança de código — lint, testes unitários, build de artefatos — a cada push. Entrega/Implantação Contínua (CD) estende isso automatizando a publicação de artefatos versionados (imagens de container, neste projeto) e, com os devidos gates de aprovação, o próprio deploy. Em um ambiente de microsserviços, isso se torna mais crítico: cada serviço tem seu próprio ciclo de release, e um pipeline mal desenhado pode publicar versões incompatíveis entre si. Boas práticas incluem: pipelines por serviço (ou matriciais, como neste projeto), uso de secrets gerenciados pela própria plataforma de CI (nunca hardcoded), artefatos imutáveis versionados por hash de commit, e gates de qualidade (testes, scan de vulnerabilidades) antes de qualquer publicação.

### 4.3 Observabilidade: métricas, logs e traces

Observabilidade é a capacidade de inferir o estado interno de um sistema a partir de seus sinais externos, e se apoia classicamente em três pilares complementares. **Métricas** são séries temporais agregadas (latência, taxa de erro, utilização de CPU) usadas para alertar e alimentar auto-scaling — no ecossistema cloud-native, o padrão de fato é o Prometheus, com um modelo de coleta pull via endpoints /metrics. **Logs** são registros discretos de eventos, e o padrão 12-Factor recomenda tratá-los como fluxos para stdout, delegando coleta, agregação e retenção à plataforma. **Tracing distribuído** reconstrói a jornada completa de uma requisição através de múltiplos serviços, correlacionando spans por um identificador de trace propagado entre chamadas (padrão W3C Trace Context) — essencial em microsserviços, onde uma falha isolada em um serviço downstream é difícil de diagnosticar olhando logs isoladamente. O OpenTelemetry se consolidou como o padrão vendor-neutral para instrumentar aplicações e exportar essa telemetria (traces, métricas e, mais recentemente, logs) para backends como Jaeger, Grafana Tempo ou soluções comerciais.

## 5. Justificativa das principais decisões arquiteturais do projeto

As decisões abaixo foram tomadas priorizando o prazo do MVP (4 semanas) e o porte médio da empresa, evitando complexidade operacional desproporcional ao problema real:

| Decisão                   | Justificativa                                                                                                                                                                                                                                   |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sem message broker no MVP | Volume e prazo não justificam a complexidade operacional de um broker; orquestração síncrona com compensação (padrão saga simplificado) resolve o fluxo atual. Ponto de extensão documentado via padrão outbox para quando o volume justificar. |
| Sem service mesh (Istio)  | Apenas 4 serviços; mTLS e traffic shaping via mesh trazem complexidade operacional desproporcional ao ganho no MVP. Métricas e tracing via OpenTelemetry já cobrem a necessidade de observabilidade avançada exigida.                           |

| Decisão                                                | Justificativa                                                                                                                                                                                                            |
|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Rolling update (não Blue-Green/Canary)</b>          | Entrega zero downtime nativa do Kubernetes, sem exigir infraestrutura duplicada (custo) nem um controlador de tráfego adicional, adequado ao prazo de 4 semanas.                                                         |
| <b>HPA por CPU, sem VPA</b>                            | VPA reinicia Pods para redimensionar recursos, o que é incompatível com um deploy verdadeiramente zero-downtime; HPA por utilização de CPU já atende ao requisito de escalar sob demanda no pico de tráfego da campanha. |
| <b>Terraform provisionando um cluster local (kind)</b> | Reprodutibilidade e custo zero para o MVP, com caminho de evolução documentado para um cluster gerenciado real (ex.: Amazon EKS), inspirado no case público de migração da iFood.                                        |

## 6. Referências e case de mercado consultado

Como exemplo concreto de mercado (fonte primária adicional exigida pelo desafio), este projeto se apoiou em dois casos públicos: (1) **Online Boutique** ([github.com/GoogleCloudPlatform/microservices-demo](https://github.com/GoogleCloudPlatform/microservices-demo)), aplicação de referência do Google Cloud com múltiplos microsserviços, Kubernetes, gRPC e observabilidade completa via OpenTelemetry, usada como benchmark de arquitetura; e (2) o **case público iFood + Amazon EKS** ([aws.amazon.com/pt/solutions/case-studies/ifood-eks-aws-case-study](https://aws.amazon.com/pt/solutions/case-studies/ifood-eks-aws-case-study)), em que uma empresa brasileira de varejo/delivery migrou sua infraestrutura Kubernetes para o Amazon EKS visando maior resiliência, redução de esforço operacional e de custos — validando a direção de evolução proposta no módulo Terraform deste projeto.

*Documentação oficial consultada: Kubernetes ([kubernetes.io/docs](https://kubernetes.io/docs) — Pods, Deployments, Services, ConfigMaps, Secrets, HPA, Pod Security Admission), Docker e Docker Compose ([docs.docker.com](https://docs.docker.com)), 12-Factor App ([12factor.net](https://12factor.net)), Terraform ([developer.hashicorp.com/terraform](https://developer.hashicorp.com/terraform)), GitHub Actions ([docs.github.com/actions](https://docs.github.com/actions)) e OpenTelemetry ([opentelemetry.io/docs](https://opentelemetry.io/docs)).*